AI Research Assistant README

AI Research Assistant — README

■

AI Research Assistant

A full-stack intelligent research platform powered by RAG, multi-model routing, and real-time web search — built as an internship assignment submission.

■ FastAPI Backend

■ RAG Pipeline

■ Web Search

■ GPT-4o

■ Pinecone VectorDB

■ PDF Intelligence

■ Table of Contents

■

Project Overview

The

AI Research Assistant

is a production-grade full-stack application that combines Retrieval-Augmented Generation (RAG), live web search, and intelligent multi-model routing into a single, elegant research interface. Users can upload PDF documents, ask natural-language questions, and receive grounded, source-cited answers from the world's best AI models.

Built entirely from scratch using Python (FastAPI), vanilla HTML/JS, and integrating with Pinecone, OpenAI, Anthropic, and Google Gemini APIs — this project demonstrates end-to-end AI application development from vector indexing to streamed response delivery.

3

AI Models Integrated

RAG

+ Web Search Pipeline

200MB

PDF Upload Limit

~2.5s

Avg Response Time

■■

Live Demo Screenshots

Welcome Screen — Research Hub

■ Welcome screen with smart prompt suggestions — RAG + Web Search context routing active

Document Analysis — RAG in Action

■ GPT-4o generating document summary with HIGH confidence rating in 2531ms — sources cited from Pinecone knowledge base

Source Cards — Retrieved Context Chunks

■ Pinecone retrieval results shown as source cards with similarity scores (0.74–0.75) — full transparency on what context was used

Session Stats Panel

■ Live token tracking — In: 666, Out: 500, Est. Cost: $0.0108

API Configuration Guide

■ Self-documenting — the assistant explains how to configure live API keys

■

Core Features

■

PDF Document Intelligence

Upload PDFs up to 200MB. The backend extracts, chunks, and indexes text into Pinecone for semantic search — ready in seconds.

■

RAG + Web Search Hybrid

Context routing combines your private document knowledge base with live web search results for comprehensive, grounded answers.

■

Multi-Model Auto-Routing

Intelligent model selector chooses between GPT-4o, Claude 3.5 Sonnet, and Gemini 1.5 Pro based on query type and complexity.

■

Source Transparency

Every response surfaces the retrieved source chunks with similarity scores — full explainability of what knowledge was used.

■

Real-time Session Stats

Live tracking of input/output tokens and estimated API cost per session. Perfect for monitoring usage during research.

■

Confidence Scoring

Each response includes a confidence level (HIGH / MEDIUM / LOW) and response latency — transparent AI quality metrics.

■

Document Summarization

One-click "Summarize Document" generates a structured overview of uploaded PDFs using the full RAG pipeline.

■

Export Chat History

Export your entire research conversation for later review, reporting, or sharing with teammates.

■

Dark Mode UI

Polished dark-first UI built in vanilla HTML/CSS/JS — no framework dependencies, fast to load, and visually professional.

■■

System Architecture

The application follows a clean client-server architecture with a dedicated RAG pipeline running on the backend.

User

Browser UI

Frontend

HTML / CSS / JS

REST calls

HTTP/JSON

FastAPI Backend

/query endpoint

/upload-pdf

/summarize

Model router

app/main.py

Pinecone

Vector Database

Semantic Search

LLM APIs

GPT-4o (OpenAI)

Claude 3.5 Sonnet

Gemini 1.5 Pro

Web Search

Live context retrieval

Embeddings Model

text-embedding-3-small

PDF Input

PyMuPDF extractor

RAG Query Flow

1

User submits a query

The frontend sends the natural language question to the FastAPI /query endpoint.

2

Query embedding generated

OpenAI text-embedding-3-small converts the query into a high-dimensional vector.

3

Pinecone semantic search

Top-K most similar document chunks retrieved from the Pinecone vector store with similarity scores.

4

Context + web search merged

Retrieved document chunks optionally augmented with live web search results based on selected routing mode.

5

LLM generates grounded response

The selected model (GPT-4o / Claude / Gemini) synthesizes a final answer from retrieved context — no hallucinations from memory alone.

6

Response + sources returned to UI

Frontend displays the answer alongside source cards, confidence rating, latency, and token usage stats.

■■

Tech Stack

FastAPI

Python web framework

Uvicorn

ASGI server

OpenAI GPT-4o

Primary LLM

Anthropic Claude

3.5 Sonnet

Google Gemini

1.5 Pro

Pinecone

Vector database

LangChain

RAG orchestration

PyMuPDF

PDF text extraction

Python-dotenv

Config management

HTML/CSS/JS

Vanilla frontend

Pydantic

Data validation

CORS Middleware

Cross-origin support

API Integrations

Service

Purpose

Model / Version

OpenAI

Chat completion + embeddings

gpt-4o, text-embedding-3-small

Anthropic

Alternative reasoning model

claude-3-5-sonnet-20241022

Google AI

Large context window tasks

gemini-1.5-pro

Pinecone

Vector storage & search

Serverless index

■

RAG Pipeline Deep-Dive

The Retrieval-Augmented Generation pipeline is the core intelligence layer. Unlike simple chatbots that hallucinate from memory, this system grounds every answer in real retrieved context.

Document Ingestion Pipeline

PDF → Chunks → Embeddings → Pinecone

def

ingest_pdf

(file_path: str):
# Step 1: Extract text using PyMuPDF
doc = fitz.open(file_path)
full_text =
" "
.join([page.get_text()
for
page
in
doc])
# Step 2: Chunk into ~500 token segments
chunks = text_splitter.split_text(full_text)
# Step 3: Generate embeddings (OpenAI)
embeddings = openai.embeddings.create(
model=
"text-embedding-3-small"
,
input=chunks
)
# Step 4: Upsert to Pinecone
index.upsert(vectors=zip(ids, embeddings, metadata))
Context Routing Modes
Mode
Description
Best For
RAG Only
Searches only the uploaded document knowledge base
Private document Q&A
Web Search Only
Performs live internet search for current information
Latest news, real-time data
RAG + Web Search
Combines both — document knowledge + live web context
Comprehensive research (default)
■
Installation & Setup
Step 1 — Navigate to backend folder
cd

ai_research_assistant/backend

Step 2 — Install dependencies

pip

install -r requirements.txt

Step 3 — Configure environment variables

Create backend/.env and add your API keys:

OPENAI_API_KEY

=sk-proj-...

ANTHROPIC_API_KEY

=sk-ant-...

GEMINI_API_KEY

=AIzaSy...

PINECONE_API_KEY

=pcsk-...

PINECONE_INDEX_NAME

=research-assistant

Step 4 — Start the backend server

python

-m uvicorn app.main:app --reload

Step 5 — Open the frontend

Open in your browser:

frontend/index.html

■

How to Use

Action

How

Notes

Upload a PDF

Click "Upload" in the sidebar

Max 200MB, auto-indexed into Pinecone

Ask a question

Type in the chat input → press Enter

Uses selected model and context routing

Summarize document

Click "Summarize Document" button

Runs full RAG pipeline over uploaded PDF

Switch model

Use "Model Selection" dropdown

GPT-4o, Claude, Gemini, or Auto-route

Change context mode

Use "Context Routing" dropdown

RAG / Web / RAG + Web

View sources

Scroll below any response

Shows retrieved chunks with similarity scores

Export session

Click "Export" in sidebar

Downloads full chat history

■

Project Structure

ai_research_assistant/

■■■

backend/

■ ■■■ app/

■ ■ ■■■ main.py

FastAPI app, all endpoints

■ ■ ■■■ rag.py

RAG pipeline logic

■ ■ ■■■ models.py

Pydantic schemas

■ ■ ■■■ utils.py

PDF extraction helpers

■ ■■■ requirements.txt

Python dependencies

■ ■■■ .env

API keys (not committed)

■■■

frontend/

■■■ index.html

Single-page UI

■■■ style.css

Dark theme styling

■■■ app.js

Chat logic, API calls

Built with ❤■ by

Sanjet Kumar

· AI Research Assistant · Internship Assignment Submission 2026

FastAPI · LangChain · Pinecone · OpenAI · Anthropic · Google Gemini